8 — Convolutional networks 1: basics

Machine Learning (MPA-MLR)

Jiří Chmelík
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Outline

1 · Shallow versus deep networks

Why three layers used to be enough — in theory — and what changed.

2 · From ANN to CNN

Local connections and shared weights, and the arithmetic behind them.

3 · Basic building blocks

Convolution, activation, pooling, fully connected, dropout and batch normalization layers.

4 · Basic CNN architecture

How the blocks are stacked — and what the first layer really learns.

1 · Shallow versus deep networks

> Two hidden layers were once considered enough for anything. They are — in the same sense in which a single very long polynomial is enough.

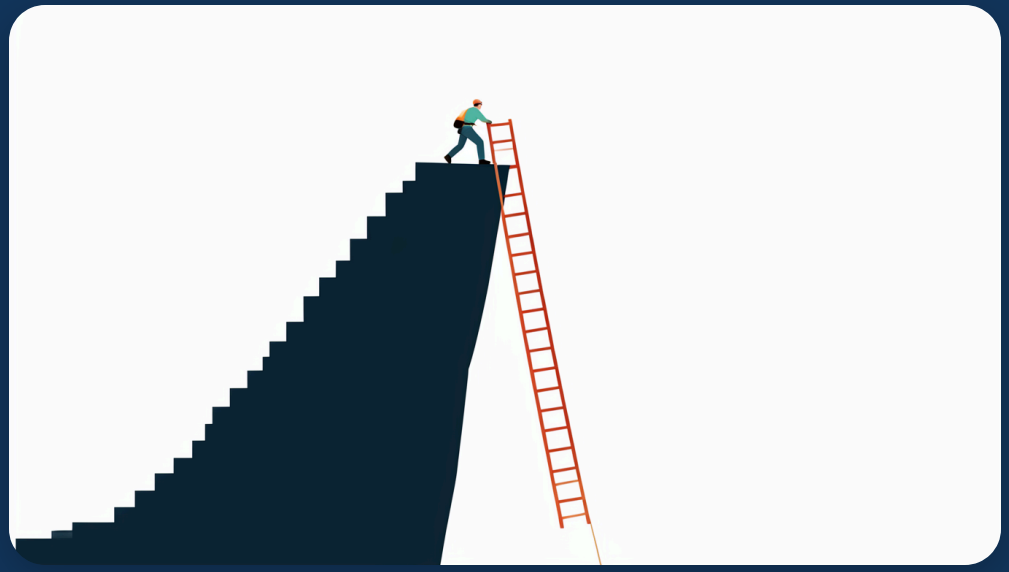
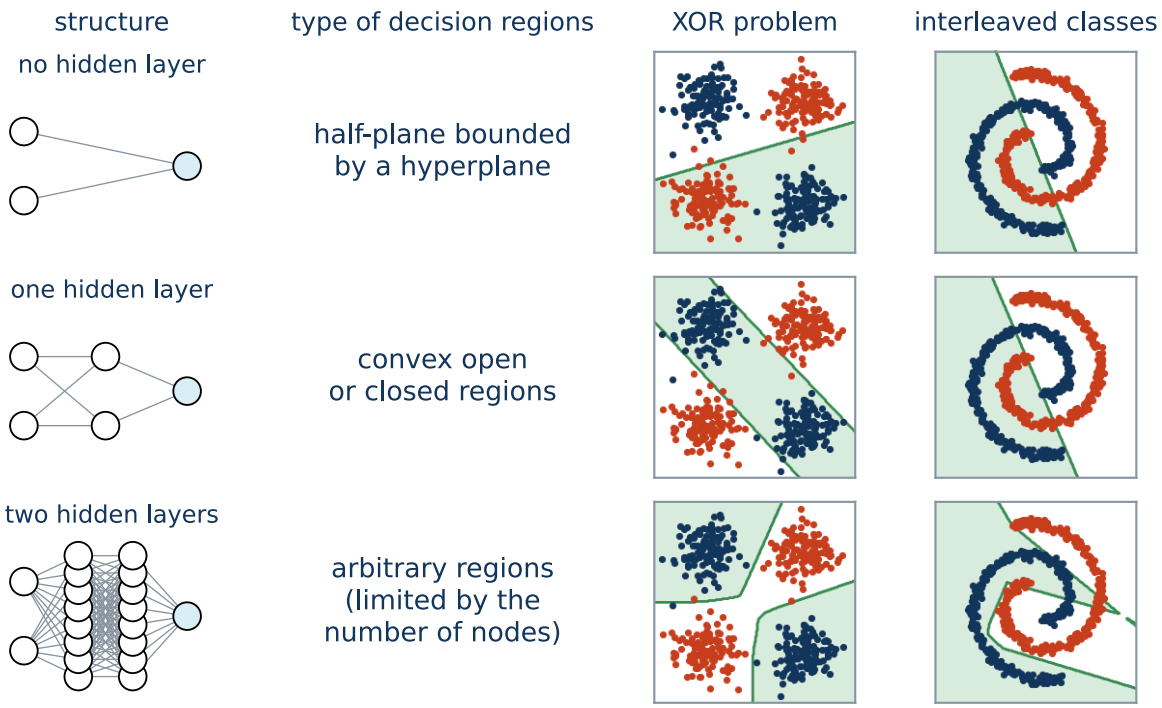


Illustration generated by AI (FLUX.2)

How many layers were once thought to be enough?

Until roughly 2006, networks used in practice were mostly **shallow**: at most three layers, that is, at most **two hidden** ones. The classical argument is about the **shape of the decision regions** a network can form (Lippmann, *IEEE ASSP Magazine*, 1987):



The two right columns are **really trained** classifiers, not drawings — see the figure generator.

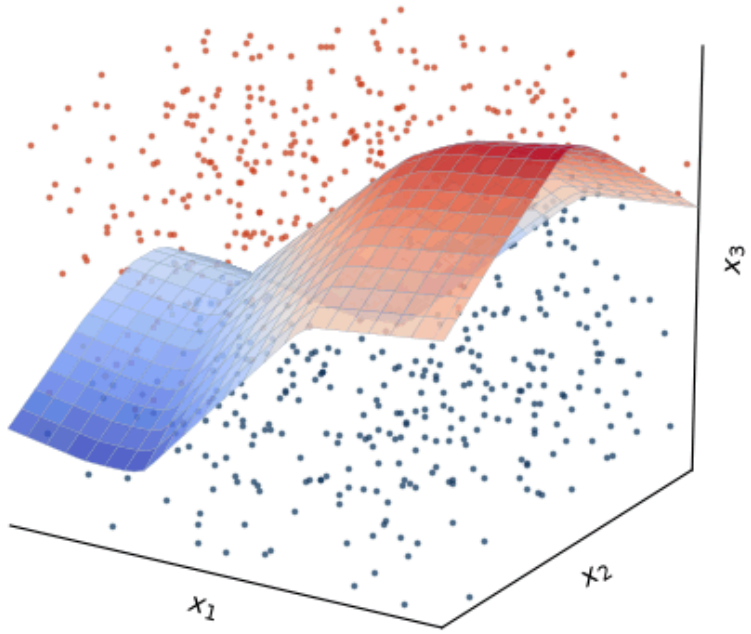
What one hidden layer can and cannot do

Universal approximation

One hidden layer with enough neurons approximates any continuous function on a compact domain (Cybenko 1989, Hornik 1991).

The catch: *enough*

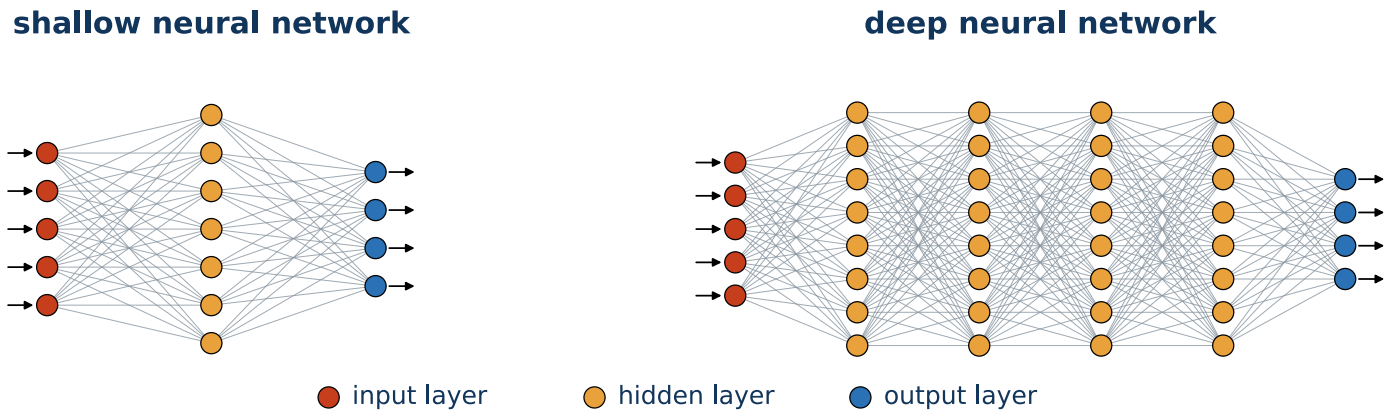
- the neuron count can grow **exponentially** with the input dimension d,
- for some functions a **deep** network needs exponentially fewer neurons (Telgarsky 2016).



The level set $\hat{p}(\mathbf{x}) = 0.5$ of a network with one hidden layer — the hyperplanes bent into a smooth surface.

Why depth pays off in practice

Deeper architectures bring a **higher level of abstraction**: low-level features formed in the first layers are combined into high-level features in the deeper ones. In practice this beats a shallow network of the same size.



> The same idea will come back twice today: the first convolutional layer detects edges, the next one corners and textures, the deepest one object parts.



Illustration generated by AI (FLUX.2)

The price of depth

✗ Huge increase in parameters

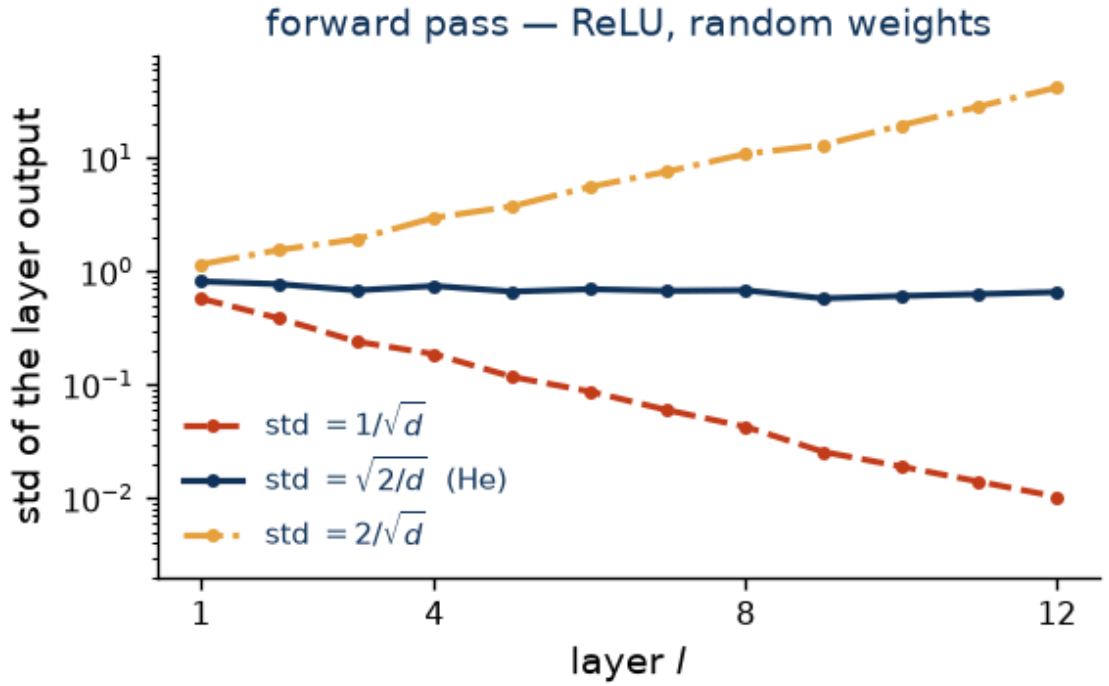
More data, memory and computation; **overfitting**.

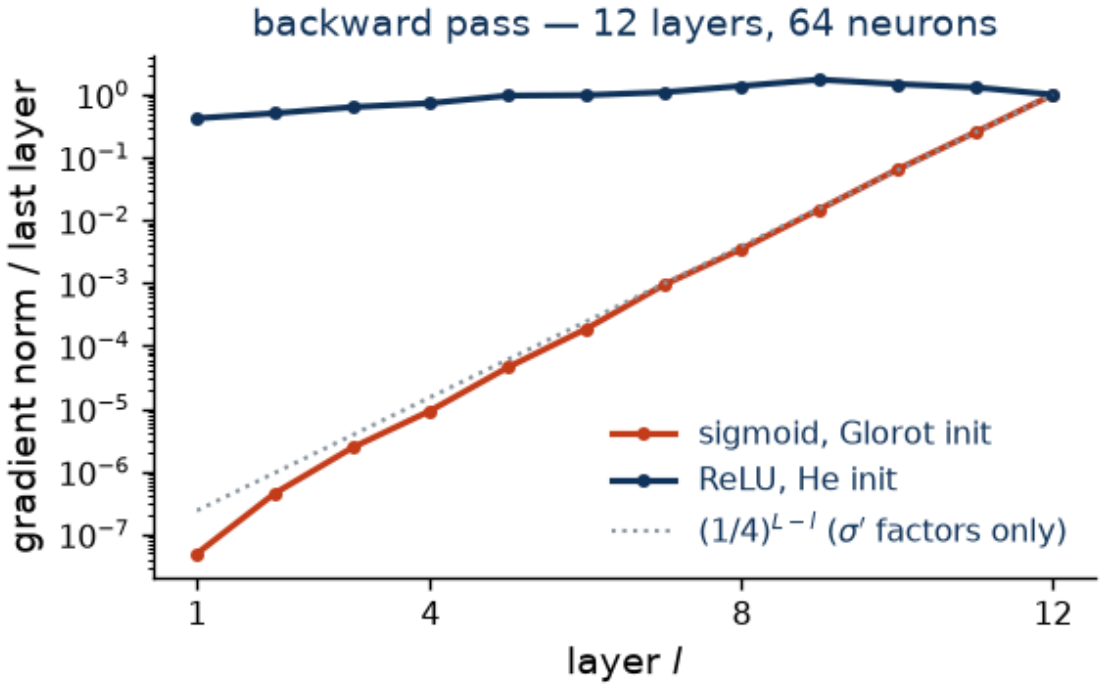
✗ Vanishing signal

$\sigma'(z) \leq \frac{1}{4}$ per sigmoid layer: ten layers $\approx 4^{-10} \approx 10^{-6}$.

Remedies

ReLU, **batch normalization**, **weight initialization** (left plot); fewer parameters: the next block.





2 · From ANN to CNN

> Fifty neurons over a 200 × 200 colour image need six million weights. Fifty 3 × 3 convolutional filters need 1 350.

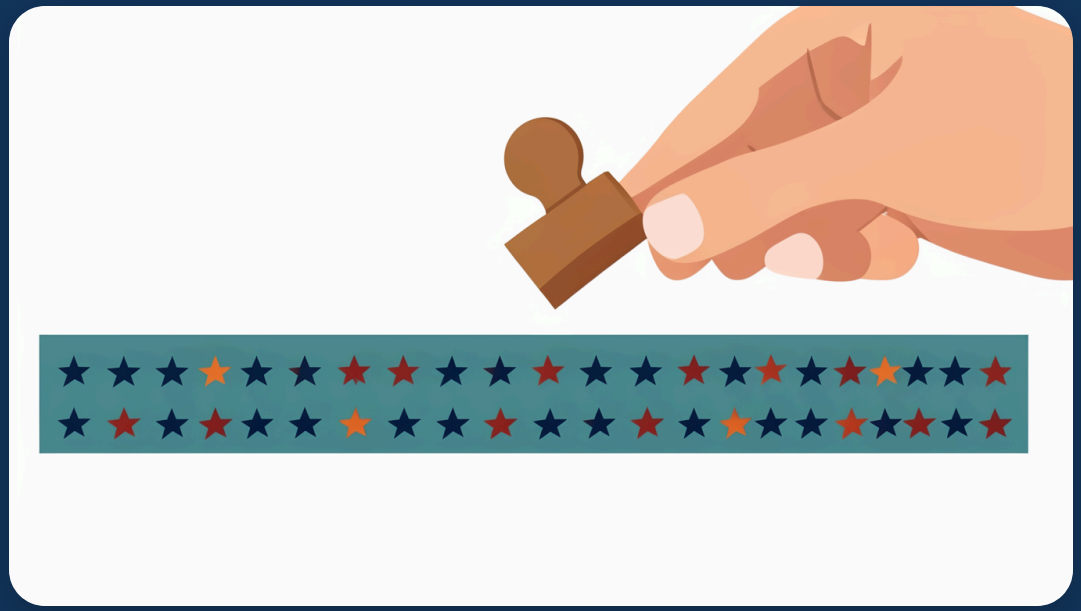


Illustration generated by AI (FLUX.2)

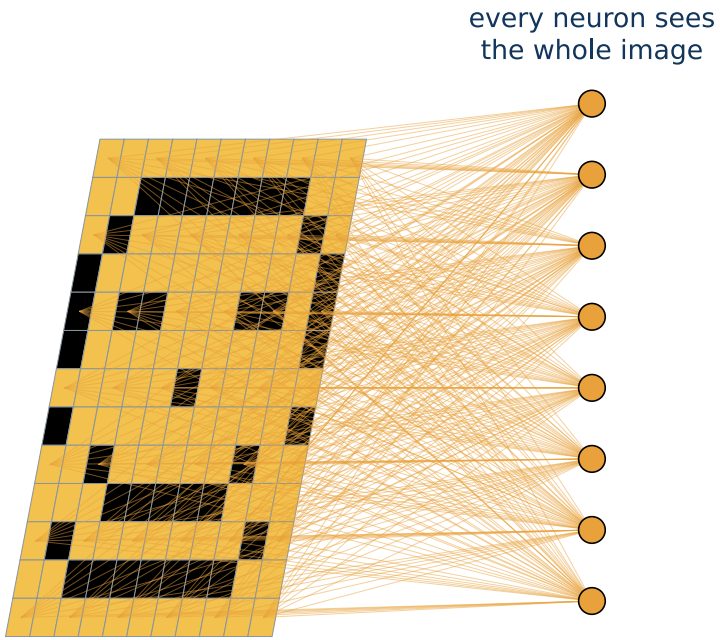
ANN — fully connected

- Each neuron is connected to **each** neuron of the previous layer.
- Each weight of each neuron is **independent** of all the others.
- No notion of **neighbourhood**: shuffling the pixels changes nothing.
- One neuron has d weights, one per input; for a colour image $200 \times 200 \times 3$:

$$d = 120\,000$$

- A layer of m neurons:

$$d \cdot m = 120\,000 \cdot 50 = 6 \cdot 10^6$$



CNN — local connections and shared weights

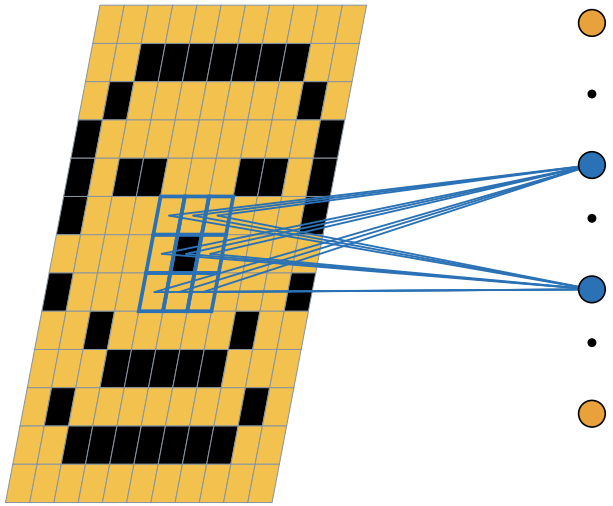
- **Local** connections: each neuron sees a small neighbourhood.
- **Shared** weights: the same feature is detected everywhere.
- One neuron — a 3×3 window over 3 channels:

$$d = 3 \cdot 3 \cdot 3 = 27$$

- A layer of K filters (here $K = 2$):

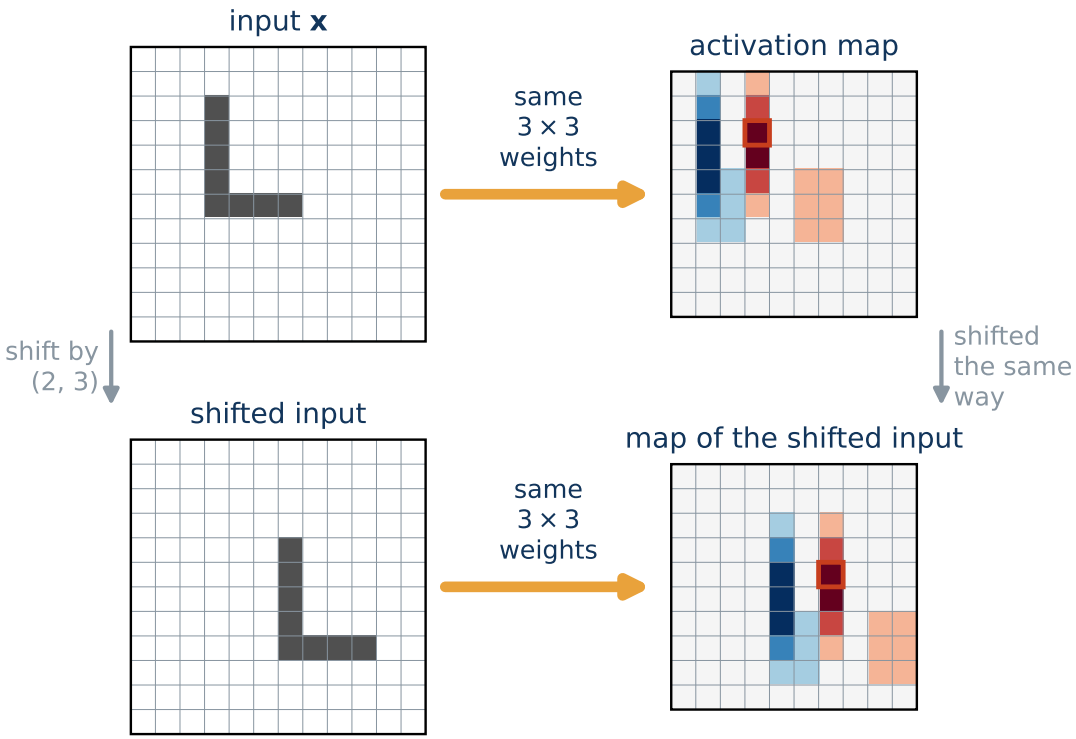
$$d \cdot K = 27 \cdot 2 = 54$$

one neuron sees one window,
all of them share the weights



Shared weights — shift the input, the map shifts

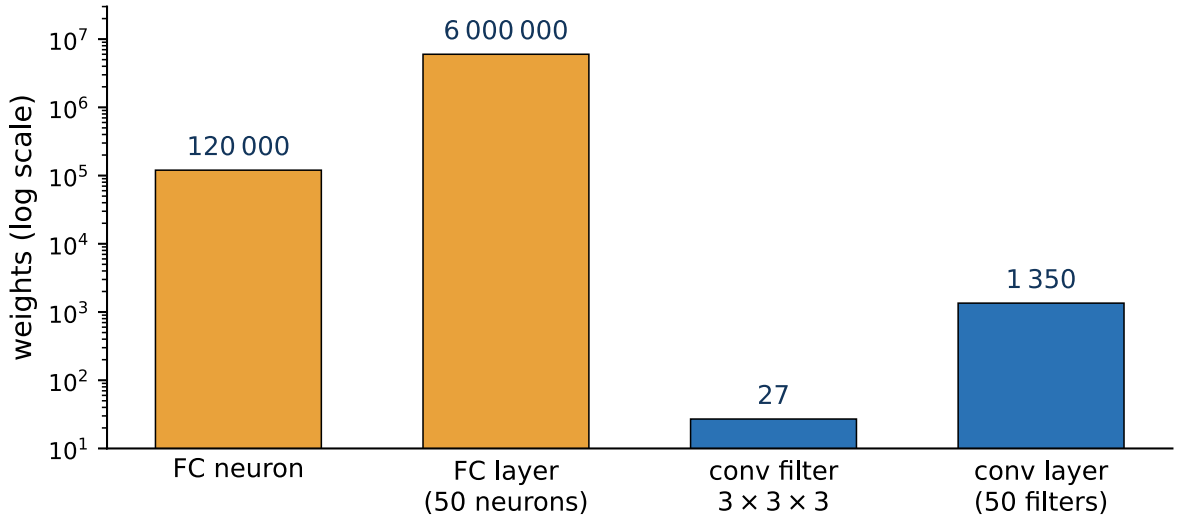
Translation equivariance: the same weights are applied at every position, so a shifted input gives the same activation map, shifted the same way — the layer learns a feature once and finds it everywhere.



Counting the weights

Weights for a $200 \times 200 \times 3$ input and 3×3 filters

	fully connected	convolutional
one neuron / filter	120 000	27
two of them	240 000	54
fifty of them	6 000 000	1 350



More than 1 000× fewer weights — 6 000 000 vs 1 350, roughly 4 400× — and **independent of the image size**.

3 · Basic building blocks

> Six layer types — the core of every convolutional network; the next lecture adds the special blocks.

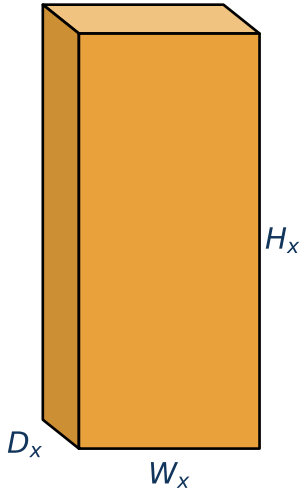


Illustration generated by AI (FLUX.2)

Convolutional layer — the input

Input image $\mathbf{x}$ of size $W_x \times H_x \times D_x$ — **width**, **height** and **depth**.

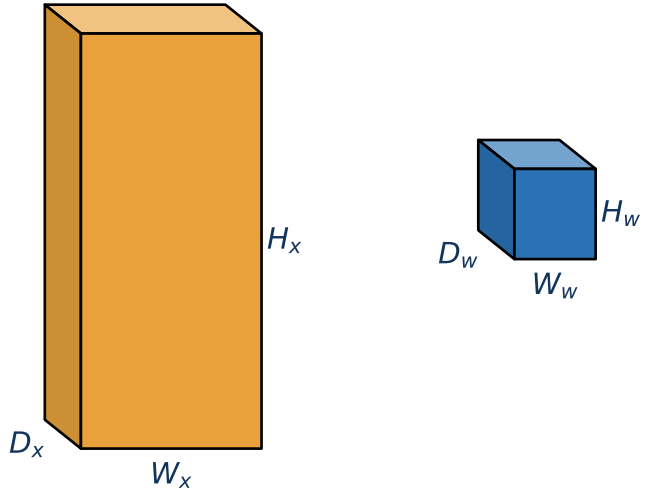
- $W_x \times H_x$ is the spatial size of the image,
- D_x is the number of **channels**: 1 for greyscale, 3 for RGB, more for a multimodal or multispectral image.



Convolutional layer — the filter

Filter $\mathbf{w}$ of size $W_w \times H_w \times D_w$, usually square, $W_w = H_w = F$.

- F is the size of the **local connection** — how big a neighbourhood one neuron sees,
- the depth equals the depth of the input, $D_w = D_x$: the filter **always** spans all channels.



Convolutional layer — one output value

The **dot product** of the filter with the part of the input under it — for filter k and the window centred at (i, j):

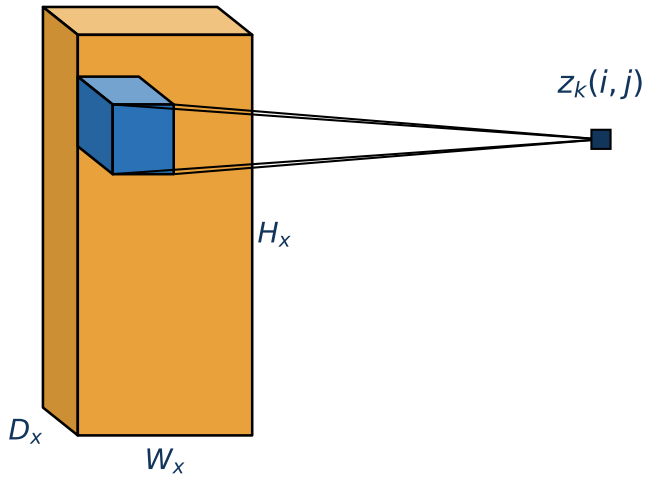
$$z_k(i,j) = \sum_{u} \sum_{v} \sum_{d=1}^{D_x} x(i+u,\ j+v,\ d)\ w_k(u,v,d)\ +\ b_k$$

with the offsets $u, v = -\frac{F-1}{2} \ldots \frac{F-1}{2}$.

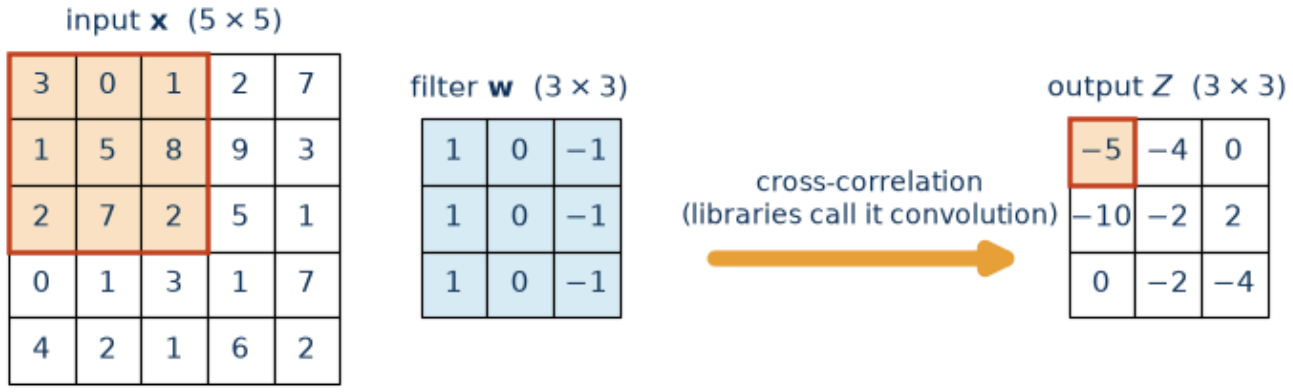
Looks familiar? Products and a sum — exactly the **inner potential** z of a neuron:

$$z_k(i,j) = \mathbf{w}_k^\top \mathbf{x}_{ij} + b_k$$

$\mathbf{x}_{ij}$ — the $F \cdot F \cdot D_x$ input values under the window, unrolled into a vector.



Convolution, worked out



$$z(0,0) = 3\cdot 1 + 0\cdot 0 + 1\cdot(-1) + 1\cdot 1 + 5\cdot 0 + 8\cdot(-1) + 2\cdot 1 + 7\cdot 0 + 2\cdot(-1) = -5$$

A vertical edge detector

Each output is the *left column minus the right column* of the window — for the first window $(3+1+2) - (1+8+2) = -5$. The output is 3×3, exactly $W_x - F + 1$ in each direction.

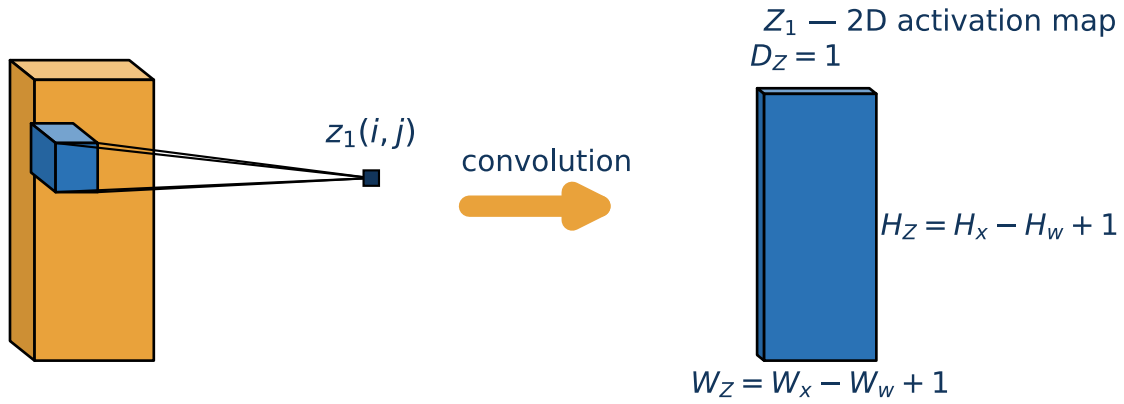
A **cross-correlation**, as in the formula on the previous slide and in every library (Conv2d): the filter is not flipped.

code example — convolution and pooling from scratch, checked against SciPy

Convolutional layer — the activation map

Sliding the filter over the whole image gives a smaller **activation map** Z_k (also called a *feature map*):

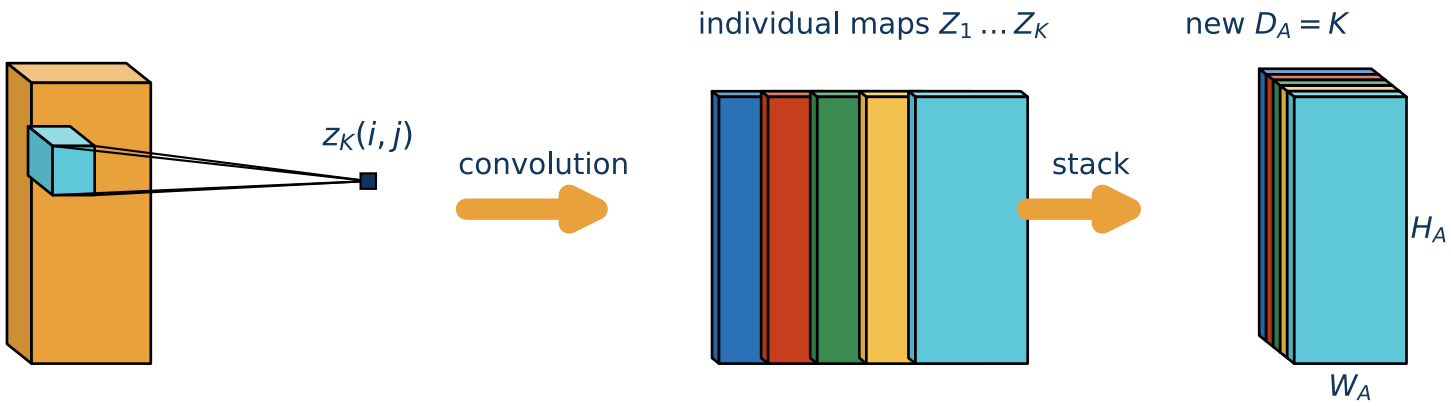
$$W_Z = W_x - W_w + 1, \qquad H_Z = H_x - H_w + 1, \qquad D_Z = 1$$



For a 32×32 image and a 5×5 filter the map is 28×28 — one filter, one map, whatever the depth of the input was.

Convolutional layer — K filters, one 3D map

Repeat the convolution with K filters — this gives K activation maps $Z_1 \ldots Z_K$, which are **stacked** into one 3D activation map A_l:

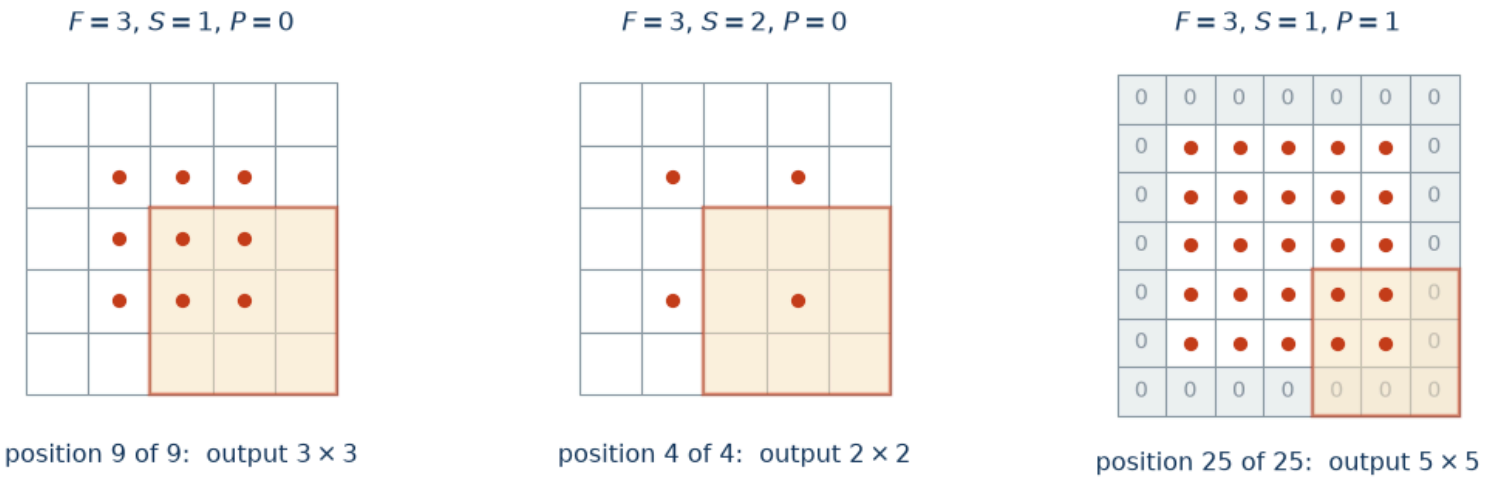


- the output of a convolutional layer is a new 3D map with $D_A = K$,
- it is the **input of the next layer** — which is why the filters of the next layer have depth K.

Notation used from here on: $z_k(i,j)$ is one number, Z_k one 2D map and A_l the whole 3D stack of layer l.

Hyperparameters of a convolutional layer

- number of filters in the layer K,
- their spatial extent F (usually an odd number),
- the **stride** S — the step of the sliding window,
- the amount of **zero padding** P — how many rings of zeros are added at the border.



Stride $S > 1$ **shrinks** the map, padding $P > 0$ **grows** it back. With $F = 3, S = 1$ and $P = 1$ the spatial size is preserved exactly — the most common setting in practice.

Output size of a convolutional layer

The input of layer l (so far $W_x \times H_x \times D_x$), $W_l \times H_l \times D_l$, becomes the output $W_{l+1} \times H_{l+1} \times D_{l+1}$:

$$W_{l+1} = \frac{W_l - F_l + 2P_l}{S_l} + 1, \qquad H_{l+1} = \frac{H_l - F_l + 2P_l}{S_l} + 1, \qquad D_{l+1} = K_l$$

input	F	S	P	K	output
$32 \times 32 \times 3$	3	1	1	16	$32 \times 32 \times 16$
$32 \times 32 \times 16$	3	1	0	32	$30 \times 30 \times 32$
$32 \times 32 \times 3$	5	3	0	8	$10 \times 10 \times 8$
$64 \times 64 \times 3$	5	2	2	8	32.5×32.5 — does not fit

Rules

- all filters in one layer must have the **same** size,
- the output size should be an **integer** in every dimension — in the last row the windows do not tile the input; PyTorch silently drops the remainder (rounds down), so check it by hand.

Playground: slide the filter yourself

input x (5 × 5)

3	0	1	2	7
1	5	8	9	3
2	7	2	5	1
0	1	3	1	7
4	2	1	6	2

filter w (3 × 3)

1	0	−1
1	0	−1
1	0	−1

output z (3 × 3)

−5	−4	0
−10	−2	2
0	−2	−4

Click an output cell to see its dot product, an input or filter cell to change it (+1; shift-click −1).

filter:

vertical edge | horizontal edge | sum | sharpen

stride **S** = 1

padding **P** = 0

next position ▸ | reset

output side = (W − F + 2P) / S + 1
= (5 − 3 + 0) / 1 + 1 = **3**

`z(0,0) = 3·1 + 0·0 + 1·(−1) + 1·1 + 5·0 + 8·(−1) + 2·1 + 7·0 + 2·(−1) =` **−5**

Convolutional layer — number of parameters

Input
3D map $W_l \times H_l \times D_l$

Output
3D map
$W_{l+1} \times H_{l+1} \times K_l$

Hyperparameters
K, F, S, P

Learned parameters
weights and biases (below)

$$\underbrace{F_l \cdot F_l \cdot D_l \cdot K_l}_{\text{weights}} + \underbrace{K_l}_{\text{biases}}$$

input	F	K	weights	biases	total
$32 \times 32 \times 3$	3	16	432	16	448
$32 \times 32 \times 16$	3	16	$2\,304$	16	$2\,320$
$16 \times 16 \times 16$	3	32	$4\,608$	32	$4\,640$
$16 \times 16 \times 32$	5	32	$25\,600$	32	$25\,632$

The number of parameters does **not** depend on W_l or H_l: one convolutional layer has as many parameters for a thumbnail as for a 4K image.

code example — output sizes and parameter counts of a whole architecture

Activation layer

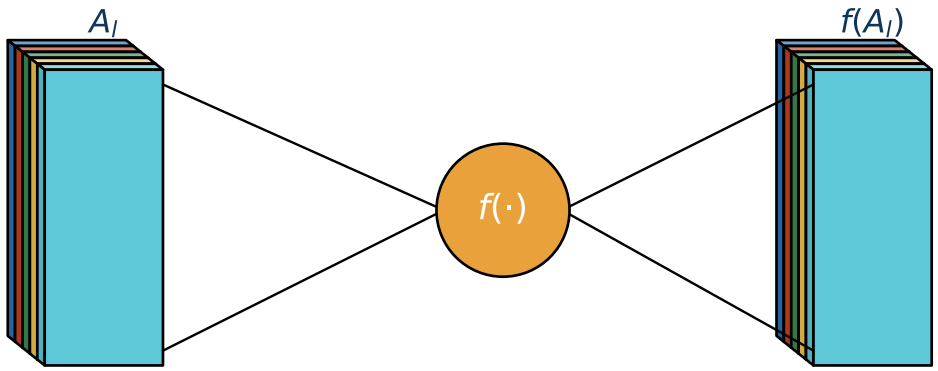
Input
3D activation map A_l

Output
3D activation map $f(A_l)$ of the **same size**

Hyperparameters
none

Learned parameters
none

- Same as in a standard neural network — an **element-wise** application of the activation function, usually **ReLU**.
- The output layer uses **softmax**, which normalizes across the classes — not element-wise.



ReLU and softmax

ReLU

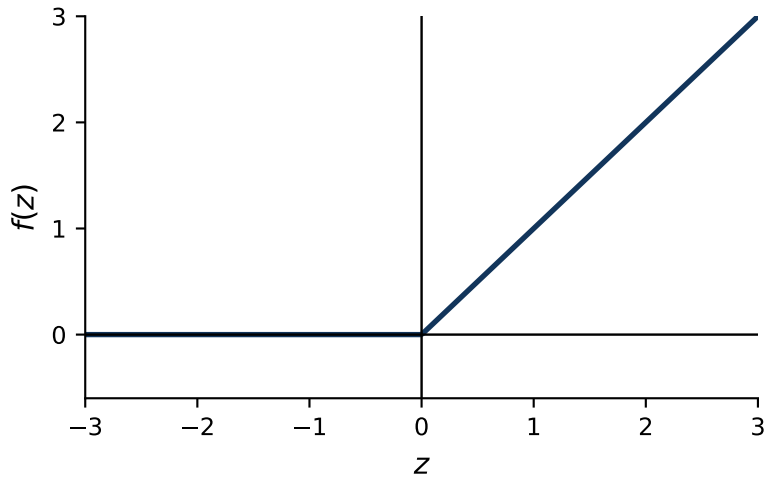
$$f(z) = \max(0, z)$$

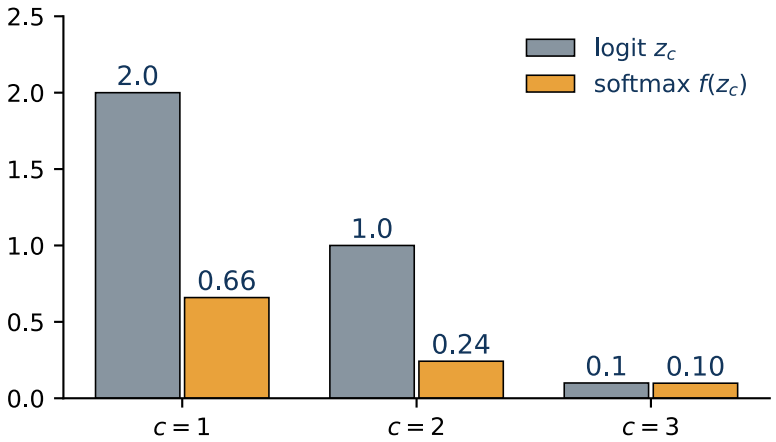
Cheap, does not saturate for $z > 0$, and its derivative is 0 or 1 — which is why gradients survive many layers.

Softmax

$$f(z_c) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}, \quad c = 1 \dots C$$

Turns the logits of the C classes into probabilities that sum to one; in a CNN classifier it sits in the output layer, together with the cross-entropy loss.





Pooling layer

Input
an image or an activation map

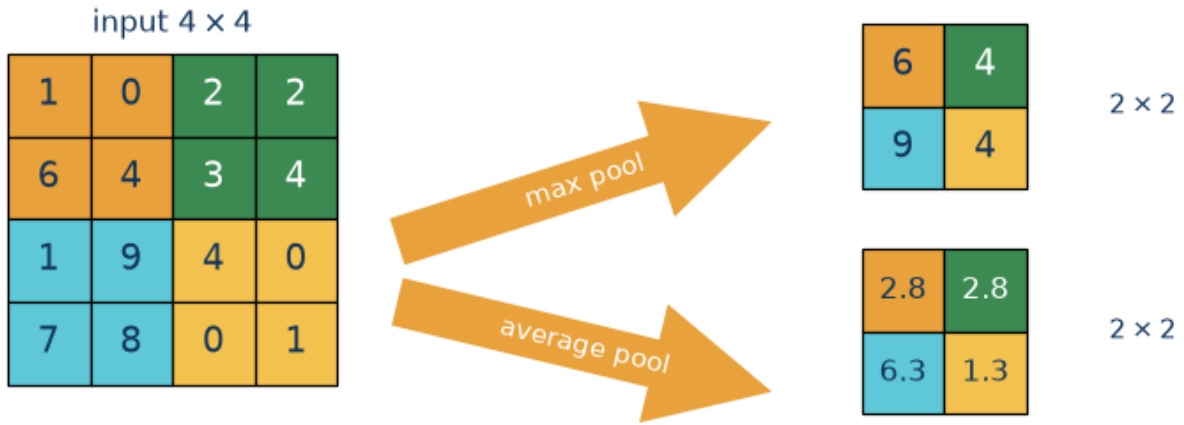
Output
shrunk in width and height; the depth unchanged

Hyperparameters
max (more often) or **average**; window F, stride S

Learned parameters
none

- Applied to each **channel separately**; it cuts the **computation** in the deeper layers and the **parameters** of the fully connected (FC) layer.

Max pooling passes on the **strongest** activation in the window: *was the feature there at all?* rather than *exactly where?*

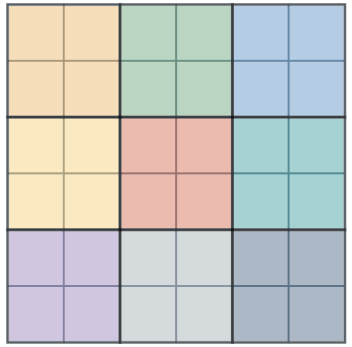


Pooling — hyperparameters and output size

Hyperparameters: the spatial extent F (usually 2 or 3) and the stride S (usually 2). Usually **no padding**; **no learned parameters**.

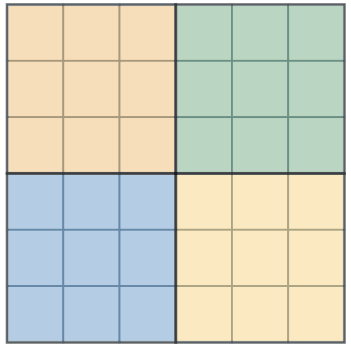
$$W_{l+1} = \frac{W_l - F_l}{S_l} + 1, \qquad H_{l+1} = \frac{H_l - F_l}{S_l} + 1, \qquad D_{l+1} = D_l$$

$F = 2$, $S = 2$



$6 \times 6 \to 3 \times 3$

$F = 3$, $S = 3$

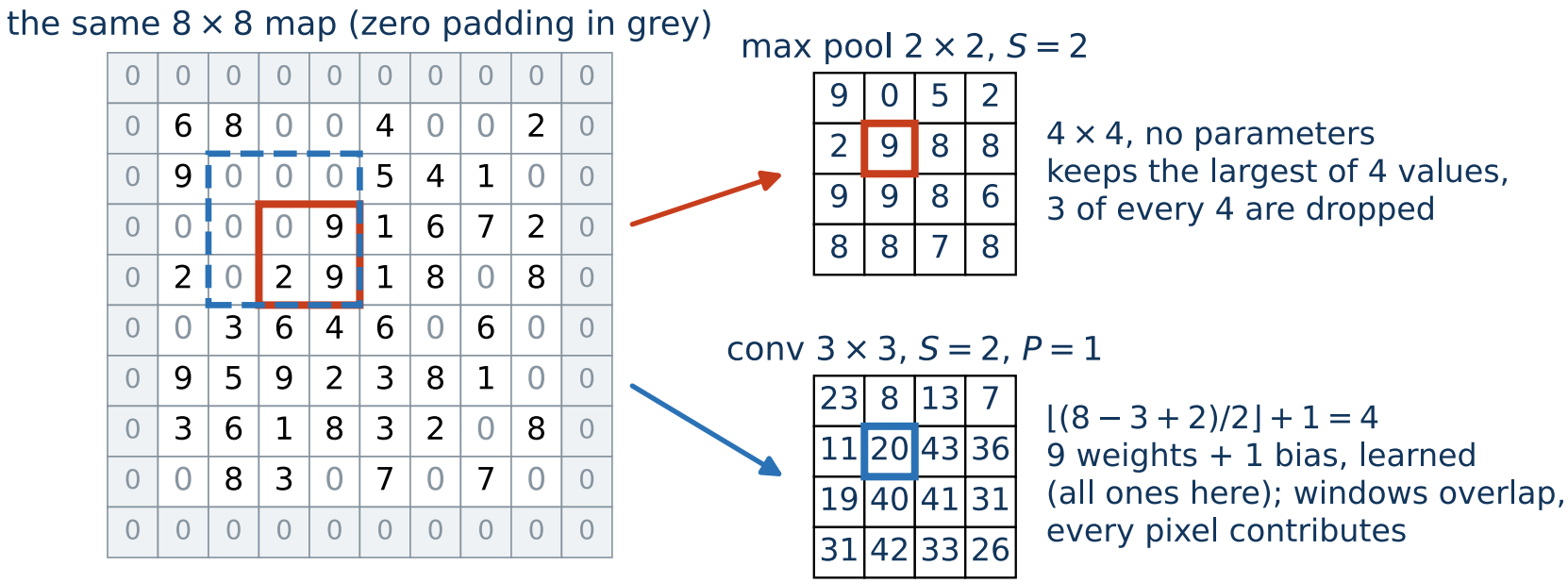


$6 \times 6 \to 2 \times 2$

- $F = S$ gives **non-overlapping** windows, the usual choice,
- $F > S$ gives overlapping windows,
- $32 \times 32 \times 16$ with $F = S = 2$ becomes $16 \times 16 \times 16$: a quarter of the values, the same depth,
- the output size should again be an **integer**.

Pooling or a strided convolution?

Both halve the map; the difference is **what is kept** and **who decides**.



Modern networks often downsample with a **strided convolution** — learned, but with parameters; the next lecture builds on it (down- and up-sampling).

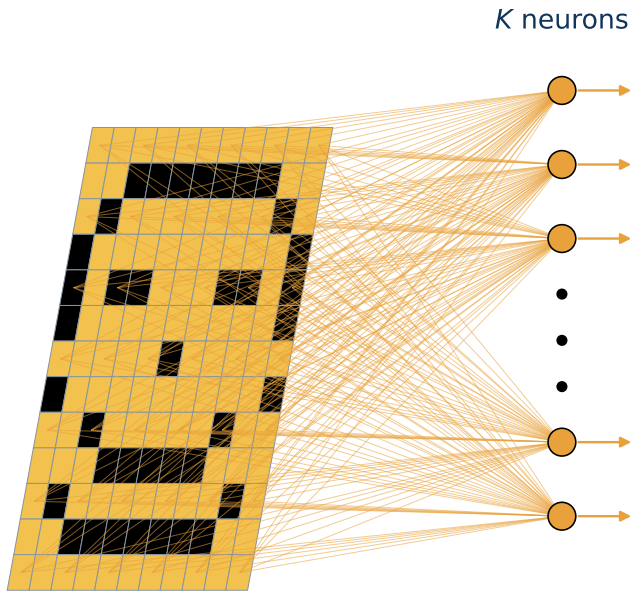
Fully connected layer

Input
a map, **vectorized** — the spatial structure is lost

Output
a vector (or a scalar)

Hyperparameters
the number of neurons K_l

Learned parameters
weights and biases (next slide)



- The **same layer** as in an ordinary neural network.
- ReLU, or softmax in the output layer.
- Turns *features* into *a decision*.

Fully connected layer — parameters

- Hyperparameter: the number of neurons in the layer K_l.
- Input size (vectorized) $W_l H_l D_l$ becomes output size K_l.
- Between two consecutive FC layers: K_{l-1} becomes K_l.
- Number of parameters:

$$W_l H_l D_l K_l + K_l$$

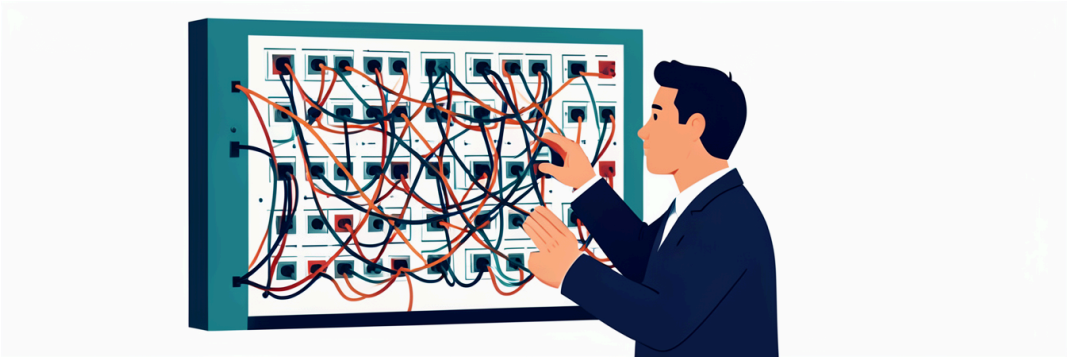


Illustration generated by AI (FLUX.2)

input	K	parameters
$8 \times 8 \times 32 = 2\,048$	64	$2\,048 \cdot 64 + 64 = 131\,136$
64	10	$64 \cdot 10 + 10 = 650$
$16 \times 16 \times 32 = 8\,192$	64	$524\,352$

One FC layer can easily cost more than all convolutional layers combined — that is why pooling shrinks the map before it.

Dropout layer

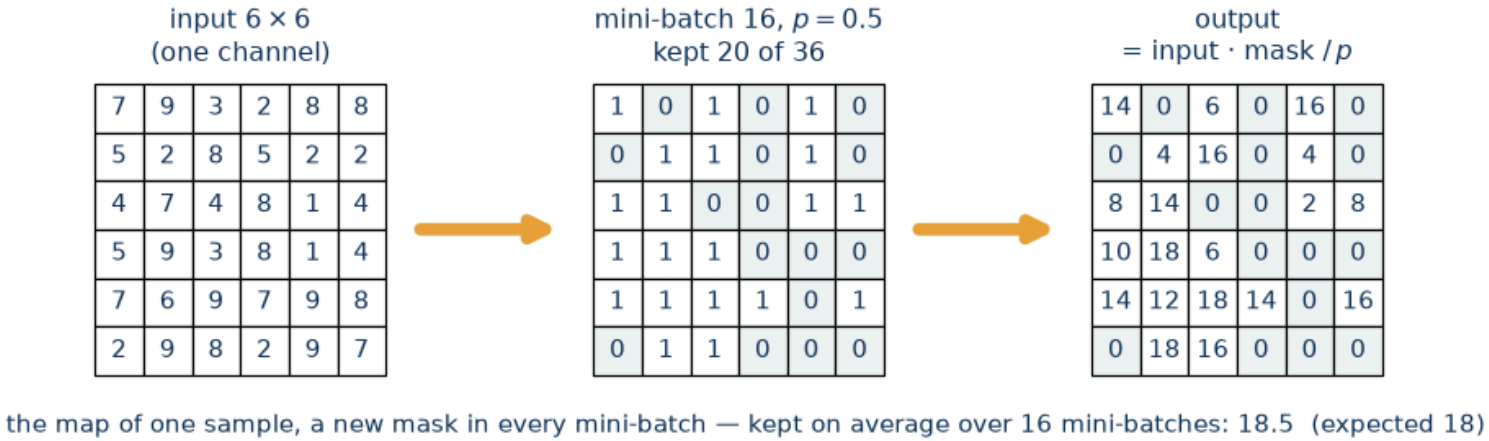
Input
outputs of a convolutional or FC layer

Output
size $W_l \times H_l \times D_l$
unchanged

Hyperparameters
p — the probability that a neuron is **kept**

Learned parameters
none

- **Randomly disables** some neuron outputs; **training phase only** — in prediction the whole network is active.
- Placed **between** layers: after the activation, before the next conv or FC.



Careful: PyTorch's `nn.Dropout(p)` uses p for the opposite — the probability that a neuron **is** dropped.

Batch normalization layer

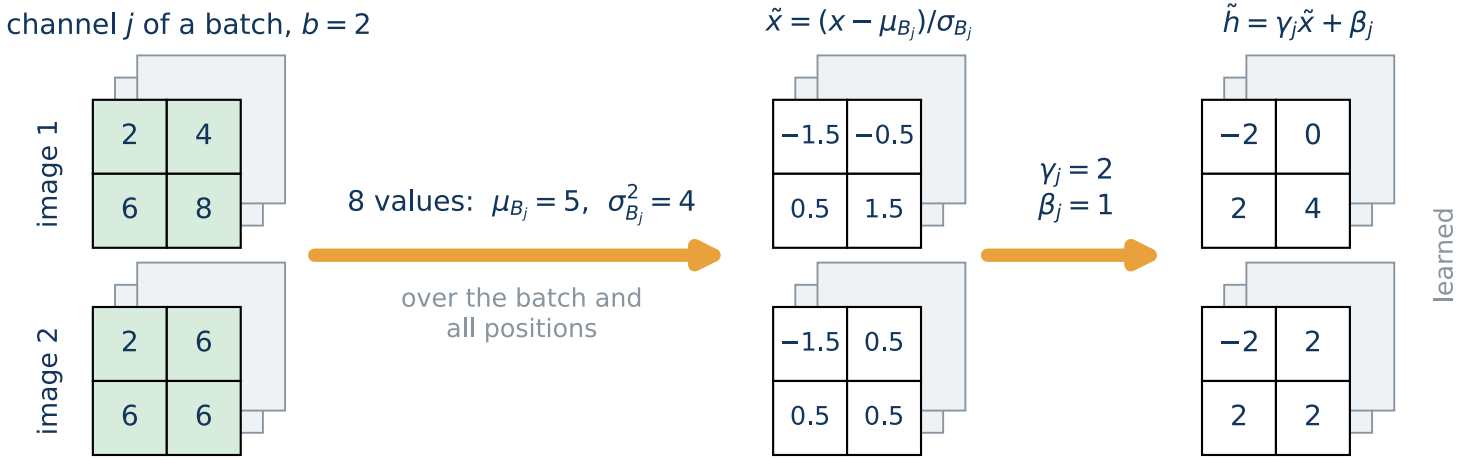
Input
the output of a conv or FC layer

Output
size $W_l \times H_l \times D_l$ **unchanged**

Hyperparameters
the decay ρ of the running mean and variance used in prediction

Learned parameters
two per channel: the scale γ_j and the shift β_j

- Normalizes to **zero mean and unit variance, per channel** j: the statistics $\mu_{B_j}, \sigma^2_{B_j}$ run over the batch **and** all positions.
- Placed **between** the convolution and its activation function.



The same layers in PyTorch


```python
import torch.nn as nn

nn.Conv2d(3, 16, kernel_size=3, padding=1)  # D=3 → K=16, F=3, P=1: 3·3·3·16 + 16 = 448
nn.ReLU()                                   # no parameters
nn.MaxPool2d(2)                             # F = S = 2, no parameters
nn.BatchNorm2d(16)                          # γ_j, β_j for 16 channels: 32 parameters
nn.Dropout(p=0.5)                           # p = probability of DROPPING (slides: keeping)
nn.Linear(8 * 8 * 32, 64)                   # 2048 · 64 + 64 = 131 136
```

slides	D	K	F	S	P
`nn.Conv2d`	`in_channels`	`out_channels`	`kernel_size`	`stride`	`padding`

The network of the table on the next slides, written this way, has exactly $148\,442$ parameters.

4 · Basic CNN architecture

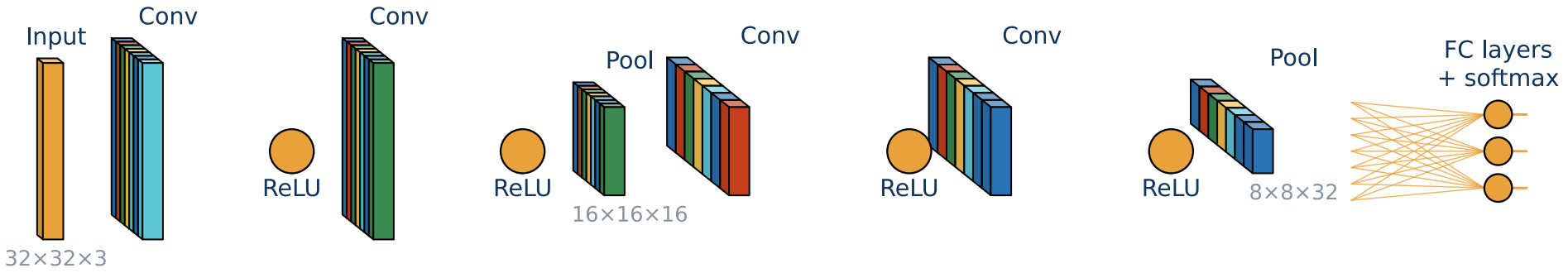
> Convolution, activation, pooling, repeat — and a fully connected layer at the very end.



Illustration generated by AI (FLUX.2)

A typical convolutional network

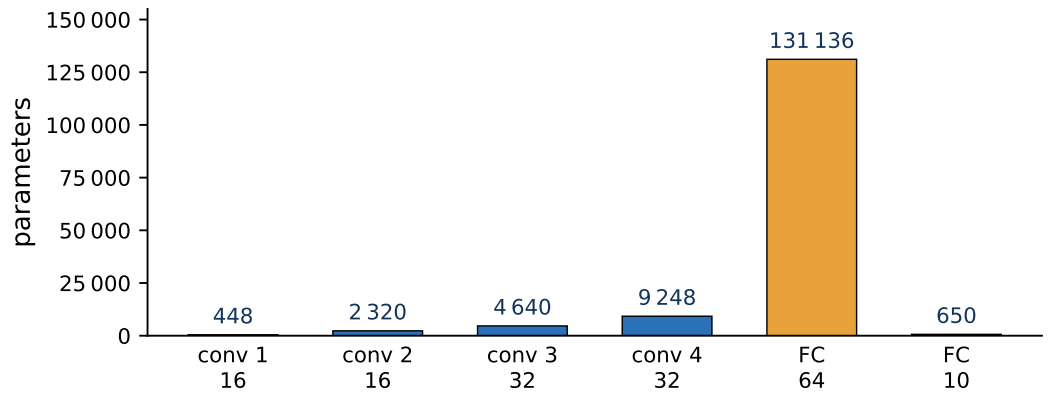
- (**conv + ReLU**) × 2–3 → **max pooling**, the block repeated a few times.
- Then **1–3 FC layers**: ReLU in all but the last, **softmax** at the output.



Width and height **shrink**, depth **grows**; at the end a 1D feature vector is classified by the output FC layer.

Sizes and parameters, layer by layer

layer	output	parameters
input	$32 \times 32 \times 3$	0
conv $3 \times 3, K{=}16, P{=}1$	$32 \times 32 \times 16$	448
conv $3 \times 3, K{=}16, P{=}1$	$32 \times 32 \times 16$	$2\,320$
max pool $2 \times 2, S{=}2$	$16 \times 16 \times 16$	0
conv $3 \times 3, K{=}32, P{=}1$	$16 \times 16 \times 32$	$4\,640$
conv $3 \times 3, K{=}32, P{=}1$	$16 \times 16 \times 32$	$9\,248$
max pool $2 \times 2, S{=}2$	$8 \times 8 \times 32$	0
FC, $K{=}64$	64	$131\,136$
FC, $K{=}10$	10	650
total		$\mathbf{148\,442}$



The four convolutional layers hold $16\,656$ parameters, the first FC layer alone $131\,136$ — **88 %** of the network.

code example — the same table computed from the layer formulas

Playground: size a network yourself

layer	output	parameters
input	32 × 32 × 3	0
conv 3×3, K = 16	32 × 32 × 16	448
conv 3×3, K = 16	32 × 32 × 16	2 320
max pool 2×2	16 × 16 × 16	0
conv 3×3, K = 32	16 × 16 × 32	4 640
conv 3×3, K = 32	16 × 16 × 32	9 248
max pool 2×2	8 × 8 × 32	0
FC, K = 64	64	131 136
FC, K = 10	10	650

total **148 442** parameters · convolutions 16 656, FC layers 131 786 · the largest layer holds **88 %**

All convolutions are 3×3 with S = 1, P = 1; pooling is 2×2 with S = 2.

input side **W** = 32

filters in block 1, $\mathbf{K_1}$ = 16

filters in block 2, $\mathbf{K_2}$ = 32

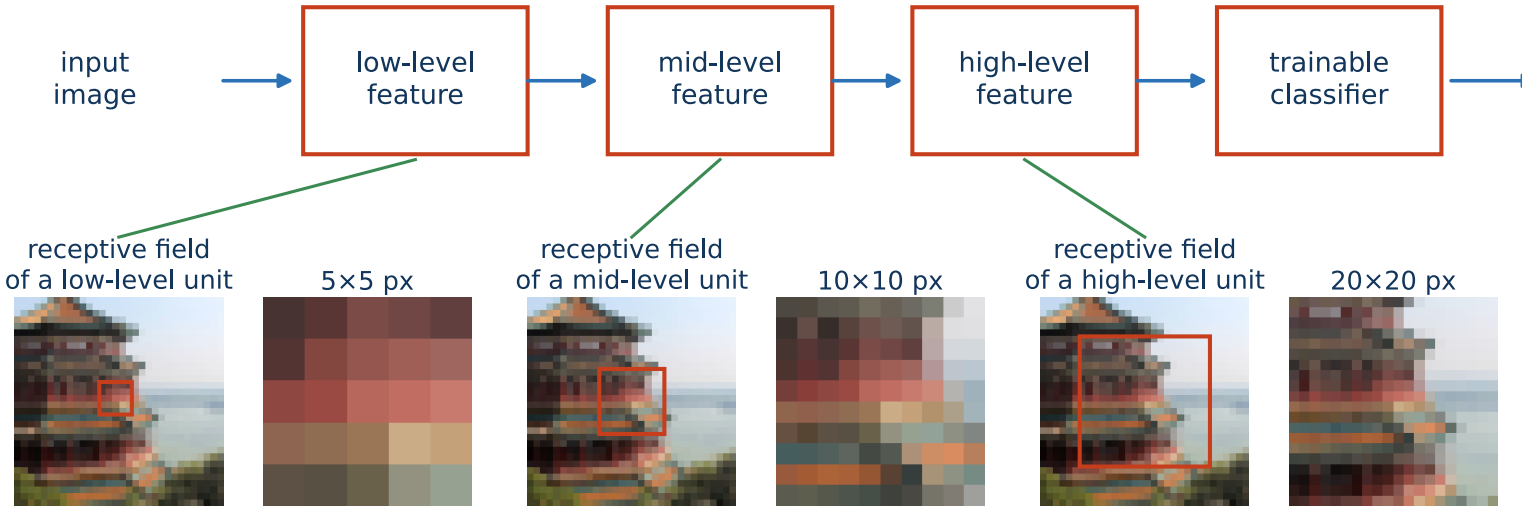
neurons of the first FC layer = 64

- [x] pooling after block 1
- [x] pooling after block 2
- [ ] global average pooling before the FC layers

the network from the table

Receptive field — what the layers see

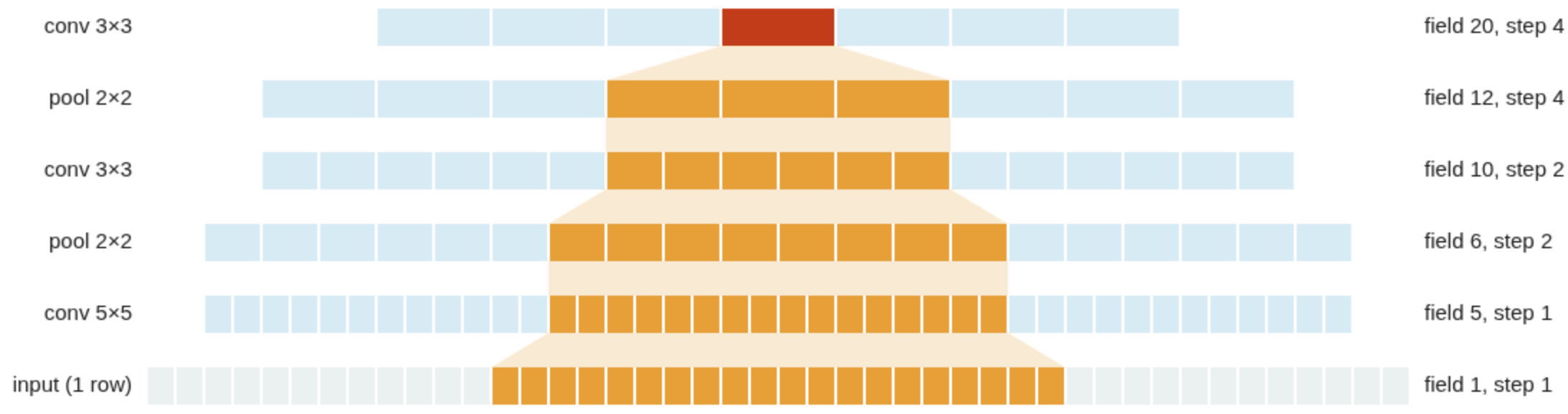
The receptive field of one neuron **grows with depth**, so deeper layers can describe larger and more abstract structures: edges, then corners and textures, then object parts.



For the stack conv $5 \times 5 \rightarrow$ pool $2 \times 2 \rightarrow$ conv $3 \times 3 \rightarrow$ pool $2 \times 2 \rightarrow$ conv 3×3 the receptive field grows to $5 \rightarrow 10 \rightarrow 20$ pixels after the three convolutions.

Layout after Zeiler & Fergus, *Visualizing and Understanding Convolutional Networks*, ECCV 2014 (arXiv:1311.2901).

Playground: how far does one neuron see?



add a layer: conv 3×3 | conv 5×5 | pool 2×2 | conv 3×3, S = 2 | remove last

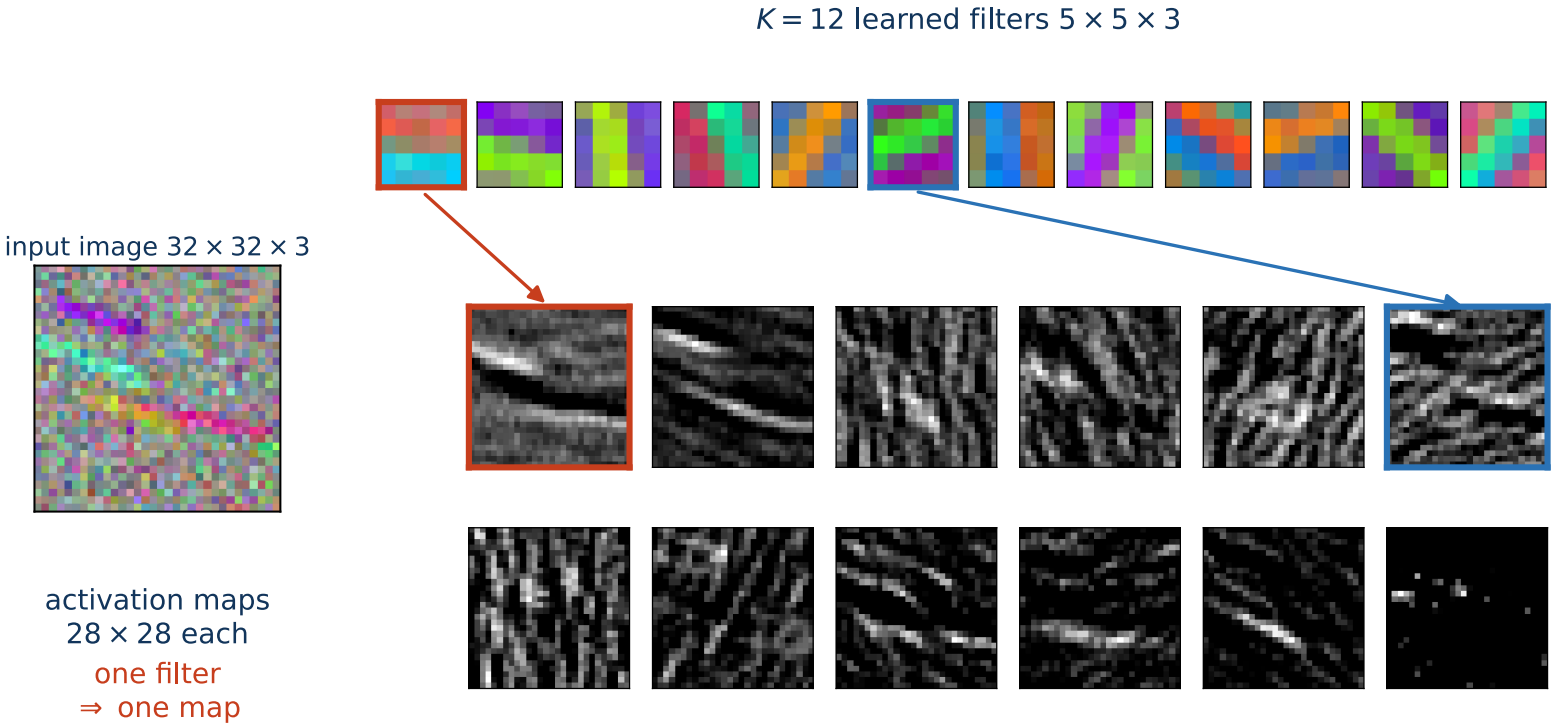
presets: the stack from the previous slide | the network from the table | two 3×3 | one 5×5

receptive field of the red unit: **20 × 20** pixels in 2D

rule: field += (F − 1) · step, step ×= S → 1 → 5 → 6 → 10 → 12 → 20

weights of the convolutions with D channels in and out: 25 + 9 + 9 = **43**·D^2

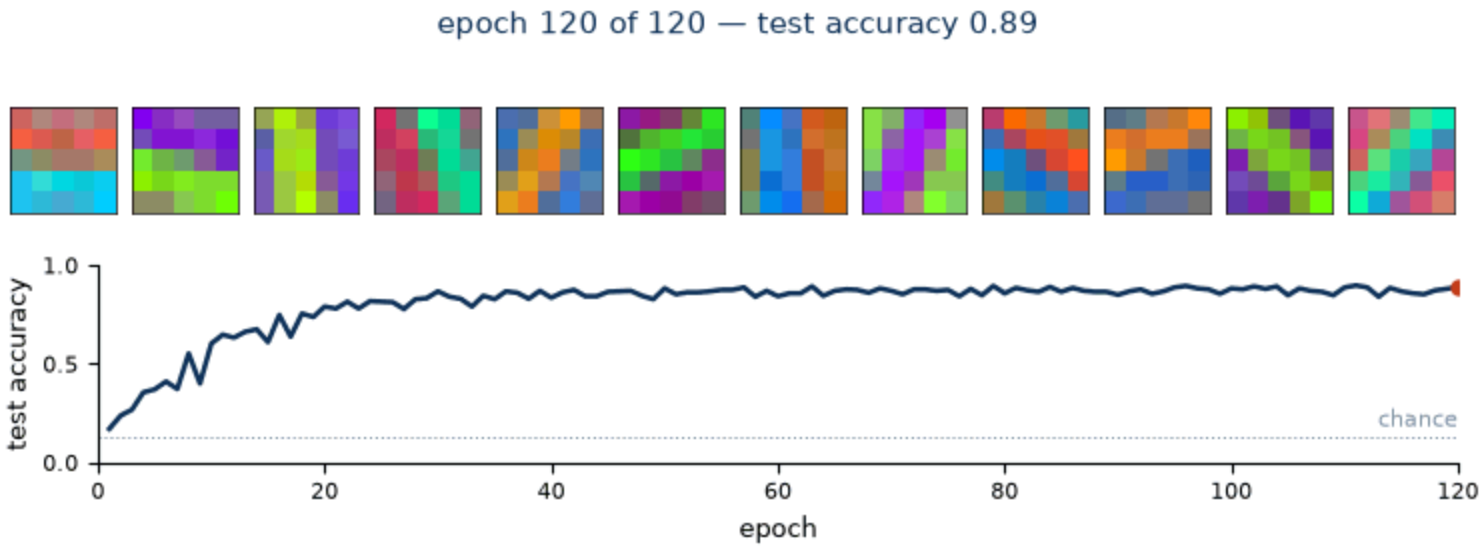
Filters and activation maps



One filter ⇒ one activation map. Trained only to classify the stroke orientation, the layer learned **oriented colour-edge detectors** by itself.

code example — this convolutional layer trained from scratch in NumPy

Where the filters come from



From **random initialization** to the end of training: the gradient of the cross-entropy loss and an L2 penalty on the weights turn noise into edge detectors; test accuracy from **near chance (1/8)** to **0.89**.

5 · Recap

> Three sets of questions: from depth, through the building blocks, to a whole network.



Illustration generated by AI (FLUX.2)

Summary

Depth

Depth buys abstraction: low-level features are combined into high-level ones. It costs parameters and brings vanishing activations and gradients.

Convolution

- A neuron with a *local* input and *shared* weights — equivariant to translation. Its number of parameters, $FFDK + K$, does not depend on the image size.
- One filter gives one 2D activation map; K filters give a 3D map of depth K, which is the input of the next layer.

Sizes

Output sizes — and the result should be an integer:

$$\text{convolution: } \frac{W - F + 2P}{S} + 1$$

$$\text{pooling: } \frac{W - F}{S} + 1$$

Other layers and the whole network

- **Pooling** shrinks width and height, keeps the depth and has no parameters.
- **ReLU** in the hidden layers, **softmax** in the output layer; **dropout** as in an ordinary network, **batch normalization** per channel, over the batch and all positions.
- A typical CNN: (conv + ReLU) × n → pooling, the block repeated a few times, then 1–3 fully connected layers — width and height shrink, depth grows.

Recap quiz I — depth and the move to convolutions

Shallow versus deep

- What limits the shape of the **decision regions** of a network with no, one and two hidden layers? What does the universal approximation theorem add — and what does it *not* promise?
- Name the two **drawbacks of depth** and one remedy for each.

From ANN to CNN

- Why is a fully connected layer a bad idea for an image? Give the **numerical** argument and the **structural** one.
- What exactly do **shared weights** mean, and what property of the layer do they give us?
- A $200 \times 200 \times 3$ image, filters 3×3, $K = 50$: how many weights does the convolutional layer have, and how many would a fully connected layer of 50 neurons have?

Recap quiz II — the building blocks

Convolution and sizes

- Write the formula for **one output value** of a convolutional layer. Over how many indices does the sum run and why?
- Why is a convolution *the same thing* as the inner potential of a neuron? How many biases does one filter have, and why only one?
- Input $64 \times 64 \times 8$, $F = 3$, $S = 1$, $P = 1$, $K = 24$: give the **output size** and the **number of parameters**.
- The same input with $S = 2$ and $P = 0$: what happens, and why is it a problem?

Pooling, dropout, batch normalization

- What does **max pooling** throw away, and why is that useful? How many parameters does it have?
- Which layers change the **depth** of the map, and which do not?
- Where would you put **batch normalization** and where **dropout**, and how does each of them behave differently in training and in prediction?

Recap quiz III — the whole network

Architecture and parameters

- Describe the **typical order** of layers in a CNN. Why does the width shrink while the depth grows?
- In the architecture from the table, which layer holds most of the parameters? What would you change to make it cheaper?

Receptive field and filters

- Two 3×3 convolutions in a row versus one 5×5: compare the **receptive field** and the **number of parameters**.
- How large is the receptive field of a neuron after conv $5 \times 5 \rightarrow$ pool $2 \times 2 \rightarrow$ conv 3×3?
- Why do the first-layer filters end up looking like **oriented edge detectors**, even though nobody designed them that way?